Project Activity 3: IPv4/IPv6 Address Application

Course DEVASC, section 4ITE
UST College of Information and Computing Sciences
John Constantine C. Acuesta
Luis Gabriel C. Araneta
Emanuel R. Ceniza
November 2026

1. Social Coding Selection

We chose Option 2. The activity asks for a prototype that shows public IP addressing to network technicians. Option 2 is a new application, so we can shape the page around the address, the ISP, the ASN, and the country code. Option 1 would add features to a directions script. This class needs the address story.

Social coding means the work is shared in Git. Each change reaches main through a pull request with one reviewer. We still have to create the GitHub repository. Until then, the link in section 4 stays a placeholder.

Address Card is the page for the demo. It is one Flask page. The map fills the screen, and the address card is a modal over that map. The same page is what we open on a phone and on a laptop.

The page has four features.

- A map that fills the screen. MapLibre draws it with the OpenFreeMap Liberty style. The map stays flat, because this app is not using Mapbox. On a laptop the card sits on the left and the pin stays in the open map. The map needs no API key.
- A modal address card. The front holds the public address, the ISP, the ASN, the country code, the city, and the IPv6 line. The back holds the roast, the coordinates, the API name, and the one ISP question. The map stays draggable while the card is open.
- An IPv6 line on the front. The browser fetches `https://api6.ipify.org?format=json`. The line reads "Checking IPv6" until the fetch returns. Failure says "No IPv6 yet".
- One roast and one ISP question. Both sit on the back of the card. The roast is one line about the ISP or the country.

On a laptop, the card tilts toward the mouse. A click on the card flips it. The X closes the card. A control labeled "Address card" opens it again.

On a phone, a tap flips the card. The X closes it. The card sits along the bottom. The phone does not tilt with a drag.

When the visitor asks for reduced motion, or when scripting is off, the card stays flat. Both faces are already in the HTML.

`fun.py` still has `map_embed` for the tests. The page does not show that iframe.

The ISP question has three choices when the ISP is known. The answer is not written into the HTML. The same ISP always rebuilds the same three choices. Grading needs no session. Address Card has no database. The lookup providers and the map need no API key.

The server fills the IPv4 fields in a fixed order. It calls ipwho.is first. If that call fails, it calls ipapi.co. Each call times out after 4 seconds. Any error becomes a notice. The route still returns status 200. Sometimes the visitor's IP is already known and both providers still fail. The page keeps that IP. The notice says that city and ISP could not be loaded. If no IP is known yet, ipify supplies the IPv4 address only. The card records the source as ipwho.is, ipapi.co, ipify, or unavailable.

The server sees the forwarded address and uses it for those IPv4 fields. The browser asks `https://api6.ipify.org?format=json` for the IPv6 line. IPv6 is not part of the server result. The line reads "Checking IPv6" until the fetch returns. A failed fetch sets the line to "No IPv6 yet".

`LOOKUP_MODE=forwarded` reads the first `X-Forwarded-For` entry and looks that address up. A missing header uses `request.remote_addr`. `LOOKUP_MODE=direct` ignores the forwarding header. In direct mode, ipwho.is reports the server's public address. That address is the home public IP. If `LOOKUP_MODE` is unset, a set `RENDER` variable selects forwarded mode. With `RENDER` unset, the mode is direct. The LAN backup can rely on that default. The backup command in section 5 still sets `LOOKUP_MODE=direct`, so the mode is obvious on stage.

Two notice sentences are fixed. When the lookup is only partly successful, the page says "The location lookup is taking a break. The address below is still yours." When no public address comes back, the page says "We could not find a public address right now. Refresh in a moment."

The layout sample, used while the page was designed, is the documentation address `203.0.113.42`. The sample ISP is PLDT, the ASN is AS9299, the country code is PH, and the city is Manila. The live page replaces those fields with the lookup.

A right answer to the ISP question shows "That's the one." A wrong answer shows "Close. This network is " and then the ISP name. When the ISP is unknown, the page shows no form. The prompt says the provider did not come back.

The activity deliverables are these.

- Address Card, the Flask page, with `app.py`, `ipinfo.py`, `fun.py`, `templates/index.html`, `static/map.js`, and `static/card.js`.
- This paper.
- The GitHub repository at [GitHub repository link], with the branches in section 4.
- The Render page at [Render live link to be added], plus a LAN backup on John's PC.
- A talk of 10 to 15 minutes, with a live demo on a phone and on a laptop.

The demo has these objectives.

- Show the visitor's public IPv4 address, ISP, ASN, country code, and city on one page.

- Show an IPv6 address, or the words "No IPv6 yet".
- Show the map pin on the lookup coordinates, and say that the pin is often the ISP's equipment.
- Show one roast about the network and one ISP question.
- Stay on the page when a provider fails, and show a notice instead of an error page.
- Reach main only through a reviewed pull request, then demo that result on a phone and on a laptop.

2. Team member roles and skillsets

Roles below are the jobs we assigned. The skillset cells are blank on purpose. We did not guess anyone's background.

Team member	Role	Assigned work	Skillset
John Constantine C. Acuesta	Scrum leader and PM	UI and pull request reviews	[team to add skillset]
Luis Gabriel C. Araneta	Recorder and QA	pytest and README review	[team to add skillset]
Emanuel R. Ceniza	Developer	API layer and lookup	[team to add skillset]

John creates the GitHub repository and reviews the pull requests named in section 4. He also builds the page. Luis keeps the task list, writes the pytest tests, and reviews the README. Emanuel builds the API layer and the lookup.

3. Strategy

We will track tasks in a Google Sheet. Luis keeps the sheet, because he is the recorder. The repo copy of that list is `tasks.csv`. We will export the sheet to `tasks.csv` so the list can sit next to the code. This paper does not paste sheet rows. The team fills the rows as the sprints run.

The build is three short sprints. Each sprint ends with something we can open.

Sprint 1 is the lookup. Emanuel will build it on the branch `emanuel/api`. The sprint is done when a request returns a card result. When a provider does not answer, the page shows a notice and stays up.

Sprint 2 is the page. John will build it on the branch `john/ui`. The sprint is done when a phone and a laptop can both show the card, the map, and the ISP question.

Sprint 3 is the tests, the docs, and the demo. Luis will add pytest on the branch `luis/tests` and review the README. John will add this paper on the branch `john/paper`. The sprint is

done when three things are true. The tests call the helper functions. The paper is in the repo. We have rehearsed the phone demo and the laptop demo.

We will build in this order.

• First, a lookup that still shows the address when a provider fails.
• Second, a page that works on a phone and on a laptop.
• Third, tests, the README, this paper, and the live demo.

The backlog in section 7 waits until those three are done. A form for an address a technician types, a header lesson, and a classmate compare are later activity work. They are outside these sprints. We also skipped two ideas that would make the demo risky. Browser location needs more JavaScript than this page can carry. A race among the three providers can fail in the middle of a live demo.

We kept these four features. They can be shown on a phone and on a laptop. They can be built before the demo. They need no API key. The page still renders when one provider is down.

Check-ins. [team to describe check-ins]

4. Using GitHub

The repository link is [GitHub repository link]. That text is a placeholder. The remote does not exist yet, so this paper has no pull request number and no real URL.

John creates an empty repository on GitHub, because he is the Scrum leader and PM. He leaves GitHub's optional README, license, and `.gitignore` unchecked. An empty repo lets the first push land without a merge. He then invites [Luis GitHub username] and [Emanuel GitHub username] as collaborators.

From the project folder, John connects the local files and pushes `main`.

```
git init
git add .
git commit -m "Start Address Card"
git branch -M main
git remote add origin [GitHub repository link]
git push -u origin main
```

The remote line above is not a working URL. Paste the https address GitHub shows, in place of [GitHub repository link], before running it. Luis and Emanuel clone that same URL after the push.

Four branches will exist. Each pull request has one reviewer. The author of a branch does not approve that same branch. John is the reviewer on `emanuel/api` and on `luis/tests`. Each

of those pull requests still has one reviewer.

Branch	Author	Reviewer	Pull request
emanuel/api	Emanuel R. Ceniza	John Constantine C. Acuesta	[PR number]
john/ui	John Constantine C. Acuesta	Luis Gabriel C. Araneta	[PR number]
luis/tests	Luis Gabriel C. Araneta	John Constantine C. Acuesta	[PR number]
john/paper	John Constantine C. Acuesta	Emanuel R. Ceniza	[PR number]

Luis reviews the README as QA. The table still names the one person who approves each pull request.

The branch flow is the same for all four. This is the lookup branch.

```
git checkout main
git pull
git checkout –b emanuel/api
```

After the lookup files are ready, Emanuel commits and pushes.

```
git add .
git commit –m "Add the lookup fallback"
git push –u origin emanuel/api
```

He then opens a pull request on GitHub from `emanuel/api` into `main`. The pull request number is [PR number] until GitHub assigns one. John is the reviewer. John approves the pull request, or he asks for a change on the same branch. Emanuel pushes any requested change to `emanuel/api`. Merge to `main` waits until John approves.

The other branches use the same commands. Only the branch name, the commit message, and the reviewer change.

- `john/ui` uses a message in the shape "Add the Address Card page". Luis reviews it.
- `luis/tests` uses a message in the shape "Add pytest for the lookup helpers". John reviews it.
- `john/paper` uses a message in the shape "Add the Activity 3 paper". Emanuel reviews it.

After a merge, the author updates local `main`.

```
git checkout main
git pull
```

If git reports a conflict, the author fixes it on the feature branch and pushes that branch again. The merge to `main` still waits for the assigned reviewer.

5. Presentation plan

The talk is planned for 15 minutes. Slides 1 through 11 take 14 minutes. Slide 12 is questions in the last minute. If the live demo runs long, we shorten slide 10 and slide 11. The backlog and the reflection are already in this paper.

Slide 1. Title. Minute 0 to 1.

The slide shows the project title, course DEVASC, section 4ITE, UST College of Information and Computing Sciences, our three names, and November 2026. The speaker says the app name, Address Card, and says the demo is live on a phone and on a laptop.

Slide 2. Chosen option. Minute 1 to 2.

The slide says Option 2. The speaker gives the reason from section 1. The class asked for a prototype that shows public IP addressing to network technicians. A new app lets us shape the page around the address, the ISP, the ASN, and the country code. Option 1 would add features to a directions script.

Slide 3. Features. Minute 2 to 3.

The slide lists the four features we are shipping.

- The map fills the screen. MapLibre draws it with the OpenFreeMap Liberty style. The map stays flat, because this app is not using Mapbox. On a laptop the card sits on the left and the pin stays in the open map. The map needs no API key.
- The address card is a modal over the map. On a laptop the card tilts toward the mouse. A click on the card flips it. On a phone a tap flips the card. The card sits along the bottom. The phone does not tilt with a drag.
- The front holds the public address, the ISP, the ASN, the country code, the city, and the IPv6 line. Failure says "No IPv6 yet".
- The back holds the roast, the coordinates, the API name, and the one ISP question.

Slide 4. Why these features. Minute 3 to 4.

The speaker says why these four stayed. They can be shown live on a phone and on a laptop. They need no API key. The page still renders when one provider is down. A typed lookup, a

header comparison, and a classmate compare wait for a later activity. Browser location and a provider race stay off the stage.

Slide 5. Architecture and APIs. Minute 4 to 5.

The speaker states the shape of the app. Address Card has no database and no API key. ipwho.is is first. ipapi.co is second. ipify supplies an IPv4 address only when the server still has none. The map fills the screen. MapLibre draws it with the OpenFreeMap Liberty style. The map stays flat, because this app is not using Mapbox. `fun.py` still has `map_embed` for the tests. The page does not show that iframe. Each lookup call times out after 4 seconds. A failure becomes a notice, and the route still returns status 200. The server sees the forwarded address and fills the IPv4 fields. The browser asks `https://api6.ipify.org?format=json` for the IPv6 line.

Slide 6. Code walkthrough. Minute 5 to 7.

The speaker names six files and what each one owns.

- `app.py` is the Flask shell. It serves the index route and `POST /quiz`. Gunicorn loads `app:app`.
- `ipinfo.py` parses the forwarding header and the provider JSON. It owns the timeout and the fallback order.
- `fun.py` owns the roast lines, the ISP question, the grading, and `map_embed`. The tests call `map_embed`. The page does not show that iframe. Those functions are pure. The tests in sprint 3 can call them without a browser.
- `templates/index.html` holds the map container, both faces of the card, and the ISP question on the back. Both faces stay in the page when scripting is off. The IPv6 fetch is a few lines in this file.
- `static/map.js` starts MapLibre with the OpenFreeMap Liberty style. It places a pin on the lookup coordinates. It loads no API key.
- `static/card.js` tilts the card toward the mouse on a laptop. A click or a tap flips the card. The X closes the card. The control labeled "Address card" opens it again. The map stays draggable while the card is open.

Slide 7. Live demo on the phone and the laptop. Minute 7 to 10.

Open the Render URL a few minutes early. Section 6 says why. Then follow this order.

1. On the laptop, drag and zoom the flat map. The card sits on the left. Then read the front of the card.
2. Move the pointer so the card leans. Click the card. Read the roast and the coordinates. Then answer the ISP question.
3. Click the X. The map remains. Click "Address card" to open it again.

4. On the phone, open the same URL. Tap the card to flip it. Tap the X to close it.

Slide 8. LAN backup. Minute 10 to 11.

If Render does not wake, start the app on John's PC. The phone joins the same Wi-Fi and opens `http://[PC LAN address]:5000`.

```
LOOKUP_MODE=direct flask run --host 0.0.0.0 --port 5000
```

Direct mode asks ipwho.is for the server's public address. That address is the home public IP. The command sets `LOOKUP_MODE=direct` so the class can see the mode. A machine outside Render already uses direct mode when `LOOKUP_MODE` is unset.

Slide 9. GitHub workflow. Minute 11 to 12.

The slide shows [GitHub repository link] and the four branch names. The speaker says we create the remote first. We invite [Luis GitHub username] and [Emanuel GitHub username]. We merge to `main` only after the one assigned reviewer approves. Pull request numbers stay [PR number] until GitHub assigns them. The full command list is section 4.

Slide 10. Backlog. Minute 12 to 13.

The speaker reads the six backlog items from section 7. They are outside the class demo. If the demo ran long, read the list and skip the comment.

Slide 11. Reflection. Minute 13 to 14.

The speaker says section 8 is a DRAFT. Each of us will replace the starter with our own words. We will not read a story that did not happen.

Slide 12. Questions. Minute 14 to 15.

Stop and ask for questions. If the class asks for a lookup we did not build, point at the backlog.

6. Live deployed link

The live page is [Render live link to be added]. The free instance sleeps, so open it a few minutes before the demo.

7. Backlog

These items are real later work. They are not part of the demo.

- Look up an address a technician types.
- Show the raw JSON for class discussion.
- Compare two addresses.

• Save a lookup to a CSV file.
• Add more roast lines written by the class.
• Look up IPv6 on the server when the host itself has IPv6.

The header lesson and the classmate compare sit with that later work. The plan calls them Activity 4 work. Browser location and a latency race are not on this list. We dropped them because they make a live demo fragile.

8. Team activities and reflection

The activities below are the plan. They are not a claim that the sprints already happened. Every reflection is a DRAFT. The team rewrites each DRAFT in their own words before the paper is final. This draft does not record a lived incident.

Planned activities.

• Luis keeps the Google Sheet task list and exports it to `tasks.csv`.
• We run the three sprints in section 3.
• Each branch in section 4 is one pull request with the reviewer named there.
• We rehearse the talk on the laptop and on the phone before class.
• Check-ins. [team to describe check-ins]

What worked well. DRAFT.

This DRAFT does not record a lived incident. The starter below only restates the plan. The work is split the way the branches are split. Emanuel takes the lookup, John takes the page, and Luis takes the tests and the README review. Each change is supposed to merge through one reviewer. Rewrite this paragraph with what the team actually saw.

Team problems and fixes. DRAFT.

This DRAFT does not record a lived incident. The team has not written a problem here. Name the problem, name the change we made, and name who made it. Delete this starter before the paper is handed in.

Technical problems and fixes. DRAFT.

This DRAFT does not record a lived incident. The team has not written a bug here. Two design facts are already in the plan, and they belong in this answer only if they show up while we build. The lookup tries ipwho.is, then ipapi.co, then a notice or ipify. The server sees the forwarded address and fills the IPv4 fields from that lookup. The browser asks api6.ipify.org, and a failed fetch sets the badge to "No IPv6 yet". Replace this paragraph with a problem that happens and with the fix.

Accountability. DRAFT.

This DRAFT does not record a lived incident. The role table in section 2 is the assignment, not a report card. Write whether each person did the assigned work. If a task stalled, write what we changed. Delete this starter if the sprints have not produced that record yet, and replace it with the real record when they have.

Decision-making. DRAFT.

This DRAFT does not record a lived incident. The plan already keeps four features. They are the 3D card, the map pin, the IPv6 badge, and one roast with one ISP question. Those four can be shown at the demo, they need no API key, and the page still works when one provider is down. The typed lookup, the header lesson, and the classmate compare stay on the backlog. Rewrite this with any decision we make that was not already written in the plan.

Team dynamics and lessons learned. DRAFT.

This DRAFT does not record a lived incident. After the sprints, write one lesson from work that happened. Leave out any story that did not happen. Delete this starter and put that lesson in its place.